



Universidade do Minho

Escola de Engenharia

Programação **I**mperativa

LETI :: 2022/23

Outubro 2022

© António Esteves

© Slides adaptados de:

CSC230 - C and Software Tools, Computer Science Faculty, Duke University, USA

BE5B99CPL – C Programming Language, Jan Faigl, Czech Technical University in Prague

Bibliografia essencial:

C Programming: A Modern Approach, capítulos 4, 5.1 e 20.1

2. Expressões e Operadores em C

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

➤ **Expressões**

➤ Operadores

- Com um operando
- Com dois operandos
- Relacionais
- Lógicos
- Atribuição

➤ Precedência dos operadores e ordem de avaliação em expressões

➤ Manuseamento de bits

➤ Operadores ao nível do bit: complemento, E, OU e OU-exclusivo

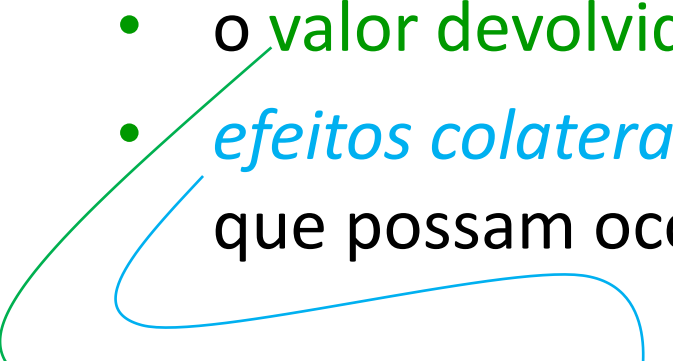
➤ Operadores para deslocamento de bits

➤ Outras operações sobre bits

- colocar bits a 0|1
- inverter os bits
- verificar se os bits estão a 0|1
- contar os bits que estão a 0|1
- copiar bits de uma posição para outra

Expressões

- A maioria das declarações num programa em C são *expressões*
- **Avaliar** uma expressão significa efetuar os cálculos de acordo com as operações nela incluídas
- **Resultado** da avaliação duma expressão:
 - o **valor devolvido** (e atribuído a uma variável); **e/ou**
 - **efeitos colaterais** (outras alterações de variáveis que possam ocorrer durante os cálculos)



```
j = k + 3 * m++;
```

➤ Expressões

➤ Operadores

- Com um operando
- Com dois operandos
- Relacionais
- Lógicos
- Atribuição

➤ Precedência dos operadores e ordem de avaliação em expressões

➤ Manuseamento de bits

➤ Operadores ao nível do bit: complemento, E, OU e OU-exclusivo

➤ Operadores para deslocamento de bits

➤ Outras operações sobre bits

- colocar bits a 0|1
- inverter os bits
- verificar se os bits estão a 0|1
- contar os bits que estão a 0|1
- copiar bits de uma posição para outra

Quais são os operadores do C?

Operador	Descrição	Associatividade
.	acesso a membro de struct ou union	esq -> dir
a[]	elemento dum vetor	
fn()	chamada de função	
++ --	inc. após , dec. após	
++ --	inc. antes , dec. antes	dir -> esq
~	negação ao nível do bit	
!	negação lógica	
- +	- , + unário	
& *	endereço , conteúdo de endereço	
(tipo)	conversão de tipo	

Operador	Descrição	Associatividade
* / %	multiplicativo	esq -> dir
+ -	aditivo	esq -> dir
<< >>	deslocamento à esquerda e direita	esq -> dir
< <= > >=	relacional	esq -> dir
== !=	(des)igualdade	esq -> dir
&	E ao nível do bit	esq -> dir
^	XOR ao nível do bit	esq -> dir
 	OU ao nível do bit	esq -> dir
&&	E lógico	esq -> dir
 	OU lógico	esq -> dir
= += -= *= /= %= &= ^= = <<= >>=	Atribuição	dir -> esq

Quais são os operadores do C?

- Há aproximadamente 50 operadores
- Categorias de operadores:
 1. "outro tipo"
 2. aritmético
 3. lógico e relacional
 4. atribuição
 5. operador ao nível do bit

Operadores de “outro tipo”

... a ver mais à frente na UC

- indexação de vetor **$x[]$**
- chamada de função **$f()$**
- operador endereço de (**$\&x$**) e conteúdo de endereço (**$*x$**)
- membro de ***struct/union*** direto (**$x.y$**) e indireto (**$x->y$**)
- operador ***sizeof()***
- operador conversão de tipo: (***tipo***)

mais à frente na UC

já falámos brevemente

mais à frente na UC

mais à frente na UC

já falámos brevemente

já falámos brevemente

- Expressões
- Operadores
 - **Com um operando**
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Operadores aritméticos com um operando ⁽¹⁾

+ unário (+a): não tem consequências

```
a = +b;
```

- unário (-b): muda o sinal do operando

```
a = -b;
```

Operadores de **incremento (++)** e **decremento (--)**

- o tipo do operando deve ser modificável, não pode ser uma constante
- estes operadores têm **efeitos colaterais**

```
a = ++b / c-- ;
```

Operadores aritméticos com um operando ⁽²⁾

++ e -- como prefixo: primeiro aplica-se o efeito colateral ao operando e depois utiliza-se o valor daí resultante na expressão

```
int i = 1, j = 8;  
printf("%d %d\n", ++i, --j);  
printf("%d %d\n", i, j);
```

O que é escrito?

++ e -- como sufixo: primeiro utiliza-se o valor antigo do operando na expressão e depois aplica-se o efeito colateral ao operando

```
int i = 1, j = 8;  
printf("%d %d\n", i++, j--);  
printf("%d %d\n", i, j);
```

O que é escrito?

02-inc-dec.c

- Expressões
- Operadores
 - Com um operando
 - **Com dois operandos**
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

- Multiplicação ($*$), quociente ($/$), resto ($\%$),
adição ($+$), subtração ($-$)
 - é possível que o resultado da avaliação da expressão ou a atribuição desse resultado não caiba na gama de representação
- Divisão por zero
 - faz com que a execução do programa falhe, se os operandos forem do tipo inteiro
 - gera um valor especial (**inf**) e a execução continua, se os operandos estiverem em vírgula flutuante

02-divideI-zero.c

02-divideF-zero.c

Operadores aritméticos com dois operandos (2)

- Os operandos da operação resto (%) **devem** ser inteiros

```
printf("%d", (5 % 3));
```

qual é o resultado?

```
printf("%d", (5 % -3));
```

qual é o resultado?

```
printf("%d", (-5 % 3));
```

qual é o resultado?

- CUIDADO: o resultado de **a%b** tem sempre o sinal do primeiro operando (**a**)
- se **b** for zero, a operação **a%b** provoca a falha do programa

02-resto.c

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - **Relacionais**
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Operadores relacionais e lógicos

- São utilizados na avaliação de condições

```
if (expressão for avaliada como VERDADEIRA)  
    ... realizar uma dada ação ...
```

- O que é VERDADEIRO em C?

- depende de como são codificados na linguagem os valores booleanos:
 - ✓ 0 representa o valor booleano FALSO
 - ✓ qualquer outro valor (1, -96, 1.414, 'F', inf) equivale ao valor booleano VERDADEIRO

```
float f = 9593.264;  
if (f) // f==VERDADEIRO ⇔ f≠0  
    ... fazer uma dada ação ...
```


Operadores relacionais (1)

- 6 operadores de comparação:

< , > , <= , >= , == , !=

```
if (a < b) ...  
if (x >= y) ...  
if (q == r) ...
```

- Os operandos devem ser números (inteiros ou reais) e o resultado é do tipo **int**
 - ↘ não se podem comparar estruturas, funções, vetores, etc.
- Se a condição for verdadeira, o resultado é **1**, senão o resultado é **0**

```
float f = 9593.264;  
if (f != 0)  
    ... fazer uma dada ação ...
```

Operadores relacionais (2)

- Um dos erros mais comuns em C é confundir:
== (comparação relacional para verificar a igualdade)
com
= (atribuição)
- Exemplo: uma estratégia de defesa num jogo ...

```
if (inimigo_lancou_ataque = confirmado)  
    retaliar();
```

não parece que esteja bem implementada, pois não?

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - **Lógicos**
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Operadores lógicos (1)

- Os operadores lógicos permitem compor condições complexas
- Os operandos devem ser (ou devolver) números (inteiros ou vírgula flutuante) e o resultado é do tipo `int`
- Operador lógico de **negação**: **!**
 - o resultado é **1** (VERDADEIRO) se o operando for **0** (FALSO), o resultado é **0** (FALSO) se o operando for **≠0** (VERDADEIRO)

```
int j = ...;  
if (! j)  
    ... faz ação quando j é FALSO ...
```

```
float f = ..., g = ...;  
if (! (f < g) )  
    ... faz ação quando f não é < g ...
```

Operadores lógicos (2)

➤ E lógico: **&&**

- avalia o valor do **primeiro** operando: se for **0**, o resultado da operação E é **0**; caso contrário
- avalia o valor do **segundo** operando: se for **0**, o resultado da operação E é **0**; caso contrário,
- o resultado da operação E é **1**

```
if (x && (y > 32) )
```

... faz ação se x é VERDADEIRO E y é > 32 ...

Operadores lógicos (3)

- Quando se avalia uma condição, ou seja, quando se calcula o valor duma condição

↘ a **ordem** dos operandos é **importante**

- Duas expressões condicionais semelhantes:

```
if ((b != 0) && ((a / b) > 5))  
    printf("quociente maior que 5\n");
```

```
if (((a / b) > 5) && (b != 0))  
    printf("quociente maior que 5\n");
```

Qual delas utilizaria? Porquê?

Operadores lógicos (4)

➤ OU lógico: `||`

- avaliar o valor do **primeiro** operando: se for **`≠0`**, o resultado da operação OU é **1**; caso contrário
- avaliar o valor do **segundo** operando: se for **`≠0`**, o resultado da operação OU é **1**; caso contrário
- o resultado da operação OU é **0**
- Não existe um OU-exclusivo lógico em C, mas ...
`(a XorLógico b) ⇔ (a && (! b)) || ((! a) && b)`

Uma ideia não recomendável

- Misturar operadores relacionais, lógicos, bit a bit e aritméticos na mesma expressão

```
unsigned char g, h;  
int          a, b;  
float        e, f;  
...  
if ((a < b) && (e * f || (g ^ h)))  
    ... fazer uma dada ação ...
```

a condição é verdadeira?

```
int    a = -4;  
char   c = 'D';  
float  e = 0.0, f = 22.2, g;  
...  
g = (c == 'D') + (e || f) * a;
```

02-misturaOps.c

qual o valor de g?

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - **Atribuição**
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Operadores de atribuição (1)

- **a = b** atribui (*copia*) o valor de **b** a **a**
 - **a** deve ser a referência a uma **variável escalar**, i.e. de um só elemento e que pode ser modificada e não pode ser nem uma função nem um vetor
- Tanto **a** como **b** devem ser um dos tipos seguintes:
 - números (inteiros ou reais)
 - **struct**'s ou **union**'s do mesmo tipo
 - apontadores para variáveis do mesmo tipo

OK

```
float a;  
int    b = 25;  
a = b;
```

errado

```
float a[2];  
int b[2] = {25, 15};  
a = b;
```

Operadores de atribuição (2)

➤ **a op= b;**

- onde **op** é um operador da lista seguinte:
***, /, %, +, -, <<, >>, &, |, ^**
- é uma forma compacta para: **a = a op b;**

```
int i = 30, j = 40, k = 50;
```

```
i += j; // o mesmo que i = i + j
```

```
k %= j; // o mesmo que k = k % j
```

```
j *= k; // o mesmo que j = j * k
```

Terminador de declaração ";" e ",",

- Normalmente, as declarações são executadas sequencialmente e estão **separadas** por **;**
- **Outro separador: ,**
 - ↘ usado por exemplo: **j = k++, i = k;**
 - 1. as expressões são avaliadas da **esquerda -> direita**
 - 2. todos os efeitos colaterais da expressão da esquerda são aplicados antes de se avaliar a expressão da direita
 - 3. o resultado final é o valor da expressão da direita
- A **","** permite uma forma **compacta** de escrever

Conteúdo

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- **Precedência dos operadores e ordem de avaliação em expressões**
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Precedência dos operadores (1)



Universidade do Minho
Escola de Engenharia

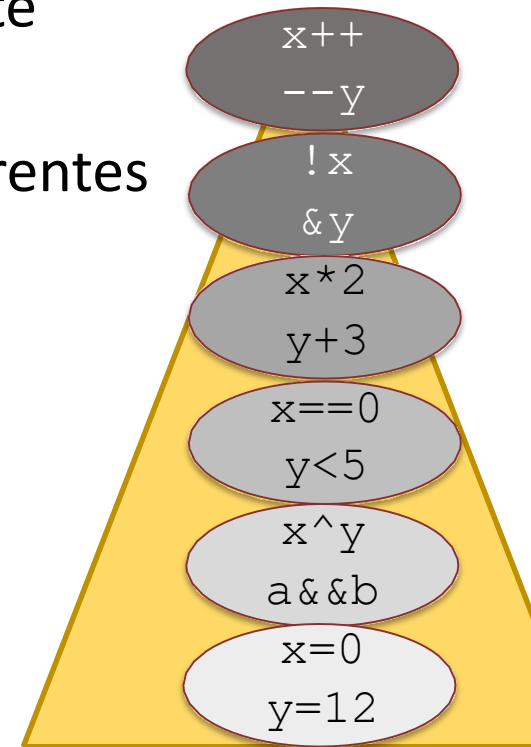
Operador	Descrição do operador	Classe	Prec.	Associatividade
a[k]	Indexação de vetor	sufixo	16	esq -> dir
f(...)	chamada de função	sufixo		esq -> dir
.	membro de	sufixo		esq -> dir
->	membro de (indireto)	sufixo		esq -> dir
++ --	incremento, decremento	sufixo		esq -> dir
++ --	incremento, decremento	prefixo	15	dir -> esq
sizeof	tamanho	unário		dir -> esq
~	negação ao nível do bit	unário		dir -> esq
!	negação lógica	unário		dir -> esq
- +	menos, mais	unário		dir -> esq
&	endereço de	unário		dir -> esq
*	conteúdo de endereço	unário		dir -> esq

Precedência dos operadores (2)

(tipo)	conversão de tipo	unário	14	dir -> esq
* / %	multiplicativo	binário	13	esq -> dir
+ -	aditivo	binário	12	esq -> dir
<< >>	desloc. esq / dir	binário	11	esq -> dir
< <= > >=	relacional	binário	10	esq -> dir
== !=	(des)igualdade	binário	9	esq -> dir
&	E bit-a-bit	binário	8	esq -> dir
^	XOR bit-a-bit	binário	7	esq -> dir
	OU bit-a-bit	binário	6	esq -> dir
&&	E lógico	binário	5	esq -> dir
	OU lógico	binário	4	esq -> dir
? :	condicional	ternário	3	dir -> esq
= += -= *= /= %= &= ^= = <<= >>=	atribuição	binário	2	dir -> esq
,	aval. sequencial	binário	1	esq -> dir

Uma forma de memorizar as precedências principais

- **Incrementos/decrementos** são extremamente aderentes
- Os outros operadores **unários** são muito aderentes
- A **aritmética** é bastante aderente
- As **comparações** são pouco aderentes
- Operações **bit a bit** e **lógicas** são muito antiaderentes
- As **atribuições** são marcadamente repelentes



- Quando não se tem a certeza **UTILIZA-SE PARÊNTESES!!!!**

➤ Exemplo:

```
a += b = q - ++ r / s ^ ! t == u ;
```

➤ Qual o valor final guardado em a?

- precisamos saber a **precedência** de cada operador
- se 2+ operadores tiverem a mesma precedência, precisamos saber a associatividade: são avaliados da **esquerda -> direita** ou da **esquerda <- direita**
 - que operação é executada em primeiro lugar, segundo lugar, ...?

Uma **solução**: usar parênteses para forçar a ordem de avaliação pretendida. Exemplo:

```
t = (u + v) * w;
```

- **Erro frequente:** ignorar as precedências e a associatividade (esq→dir ou esq←dir)

```
t = u+v * w;
```

Recomendação:

- em caso de dúvida, forçar a ordem da avaliação **utilizando parênteses**

ou

- melhor ainda, escrever uma expressão complexa como uma sequência de várias **expressões mais simples**

➤ Em vez de ...

```
a+=b=q-++r/(s^!t==u);
```

ou de ...

```
a+=(b=(q-((++r)/(s^((!t)==u)))));
```

optar por ...

```
tmpBool = ( (!t) == u );  
tmp2 = s ^ tmpBool;  
tmp3 = (++r) / tmp2;  
b      = q - tmp3;  
a      += b;
```

Conteúdo

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- **Manuseamento de bits**
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Rever a representação hexadecimal

Dígito hexadecimal	Binário	Decimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15

0xDEADBEEF
1101 1110 1010 1101 1011 1110 1110 1111

0x02468ACE
0000 0010 0100 0110 1000 1010 1100 1110

0x13579BDF
0001 0011 0101 0111 1001 1011 1101 1111

Manuseamento de bits (1)

- O C dispõe de operadores que tratam os operandos como **sequências de bits**
- Questão: Qual é o interesse em efetuar operações ao nível do bit?
- Resposta 1: por vezes isso permite reduzir o espaço ocupado pelos dados
- Resposta 2: há algumas operações de processamento de dados que são mais rápidas ao nível do bit do que com operações aritméticas

Manuseamento de bits (2)

➤ Exemplo: processamento de imagens

- juntar 64 pixéis numa imagem a preto e branco numa única variável do tipo **long long** e processar esses 64 pixéis apenas com uma instrução
- aplicar um filtro a uma imagem de modo a criar outra camada sobreposta à imagem original

➤ Outros exemplos de aplicação:

- compressão de dados
- cifrar dados
- correção de erros
- interação com dispositivos de entrada/saída
- ...

Trabalhar com valores binários em C?

➤ Não há uma forma normalizada de ...

- **definir** uma constante em binário

```
i = 01011011;
```

- **receber** do utilizador uma *string* contendo 0's e 1's e convertê-la para um número inteiro

```
scanf("%b", &i);
```

- **escrever** no terminal um número inteiro utilizando o formato binário (com caracteres 0 e 1)

```
printf("%b", i);
```

➤ Quais as alternativas?

- Utilizar a representação **octal** ou **hexadecimal**

Conteúdo

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- **Operadores ao nível do bit** - complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Representação de números inteiros negativos

- **Complemento para 1** de um número: os zeros passam a uns e os uns passam a zeros
- **Complemento para 2** de um número: é igual ao complemento para 1 mais **um**

bit

0x0d

Complemento para 1

7 6 5 4 3 2 1 0

0 0 0 0 1 1 0 1

1 1 1 1 0 0 1 0

+1

Complemento para 2

1 1 1 1 0 0 1 1

trocar bits

somar 1

com 4 bits

Binário	Decimal	Complemento para 1	Complemento para 2
0000	0	1111 = -0	0000 = 0
0001	1	1110 = -1	1111 = -1
0010	2	1101 = -2	1110 = -2
0011	3	1100 = -3	1101 = -3
0100	4	1011 = -4	1100 = -4
0101	5	1010 = -5	1011 = -5
0110	6	1001 = -6	1010 = -6
0111	7	1000 = -7	1001 = -7
1000	8	0111	1000 = -8

Operadores ao nível do bit: com um operando

➤ **Complemento** ao nível do bit: $\sim$

- o operando tem que ser um inteiro
- o resultado é o **complemento para 1** do operando: os 1's passam a 0's e os 0's passam a 1's

- exemplo: $\sim 0x0d$ // $0x0d$ em binário é **00001101**
 $= 0xf2$ // $\sim 0x0d$ em binário é **11110010**

➤ Não é o mesmo que a **negação lógica (!)** ou a **mudança de sinal (-)**:

complemento para 2

```
char i, j1, j2, j3;  
i = 0x0d; // binário é 00001101  
j1 = ~i; // binário é 11110010  
j2 = -i; // binário é 11110011  
j3 = !i; // binário é 00000000
```

Operadores ao nível do bit: com 2 operandos ¹

- Fazem operações **bit a bit** entre os operandos, produzindo um resultado com o mesmo número de bits dos operandos
- **& (E)**: o bit **i** do resultado só é 1 se ambos os bits na posição **i** dos operandos forem 1, é 0 nos outros casos
- **| (OU)**: o bit **i** do resultado só é 0 se ambos os bits na posição **i** dos operandos forem 0, é 1 nos outros casos
- **^ (OU exclusivo)**: o bit **i** do resultado é 1 quando os bits na posição **i** dos operandos forem diferentes um do outro (0-1, 1-0), é 0 quando forem iguais (0-0, 1-1)
- Os operandos têm que ser **inteiros**

Operadores ao nível do bit: com 2 operandos₂

➤ Exemplos:

	0011	1000
&	1101	1110
	0001	1000

	0011	1000
	1101	1110
	1111	1110

	0011	1000
^	1101	1110
	1110	0110

Diferença entre operações lógicas e bit a bit

Quais os resultados? Quais as diferenças? Há problemas?

```
int a, b, c,  
    d, e, f;  
  
int i = 30;  
int j = 0;  
a = i && j;  
b = !j;  
c = !i;  
  
float x = 30.0;  
float y = 0.0;  
d = x || y;  
e = !y;  
f = !x; // 02-logicOps.c
```

```
int a, b, c,  
    d, e, f;  
  
int i = 30;  
int j = 0;  
a = i & j;  
b = ~j;  
c = ~i;  
  
float x = 30.0;  
float y = 0.0;  
d = x | y;  
e = ~y;  
f = ~x; // 02-bitOps.c
```

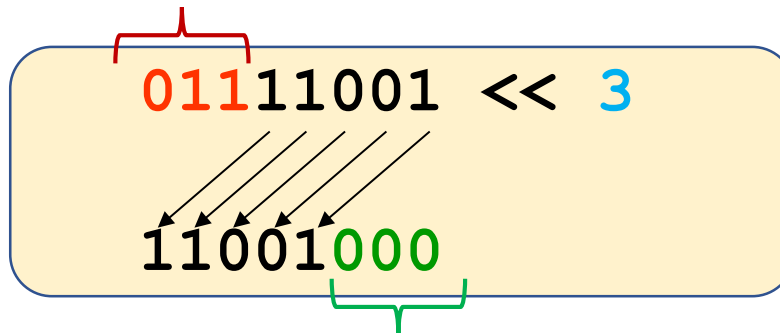
- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- **Operadores para deslocamento de bits**
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Operadores para deslocamento de bits ⁽¹⁾

➤ **x** << **y** é o deslocamento à esquerda (lógico) de **x** em **y** posições

- **x** e **y** devem ser ambos números inteiros
- **x** deve ser **unsigned int** ou **int** positivo
- faz sentido que $0 \leq \mathbf{y} \leq \text{número_de_bits_de_}\mathbf{x}$
- os **y** bits mais à esquerda de **x** são descartados
- são inseridos **y** zeros nos bits mais à direita

estes 3 bits são descartados

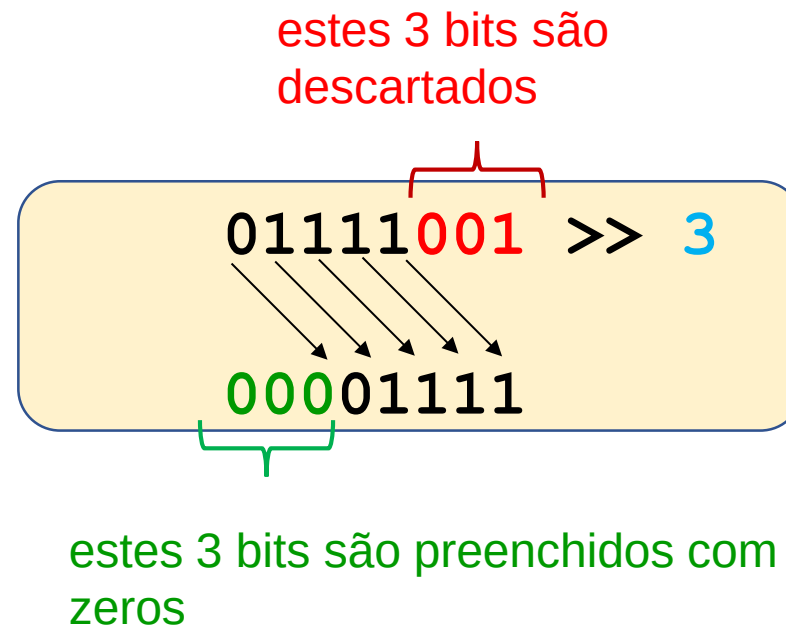


estes 3 bits são preenchidos com zeros

Operadores para deslocamento de bits (2)

➤ **x** >> **y** é o deslocamento à direita (lógico) de **x** em **y** posições

- os **y** bits mais à direita de **x** são descartados
- os **y** bits mais à esquerda de **x** são preenchidos com zeros



Operadores para deslocamento de bits (3)

➤ Por vezes ...

- o deslocamento lógico à direita de um número x em y posições é utilizado para dividir x por 2^y
- o deslocamento lógico à esquerda de um número x em y posições é usado para multiplicar x por 2^y

```
unsigned short j, k, m;  
j = 121;  
k = j << 3;  
m = j >> 3;  
printf("%d %d %d\n", j, k, m);
```

```
// 02-shiftBits.c
```

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - **colocar bits a 0|1**
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Outras operações sobre bits

- Calcular o complemento, o E, o OU e o OU exclusivo ao nível do bit são operações executadas diretamente pelos operadores do C
- E as operações seguintes são possíveis?
 - colocar a 0|1 determinados bits no operando **x**
 - verificar se determinados bits de **x** são 0|1
 - contar o número de bits a 0|1 no operando **x**
 - copiar 1+ bits da posição **i** de **x** para a posição **j** de **y**

Outras operações: colocar bits com valor 0₍₁₎

- Utilizando o operador **&**:
 - Efetuar a operação **E (&)** entre o operando e '0' coloca os bits do operando a 0, enquanto o **E** entre o operando e '1' não altera os bits do operando
- Ou seja, definimos uma máscara (padrão de bits) com 0's nas posições onde queremos colocar o operando a 0 e 1's nas outras posições

bit do operando	bit da máscara	operando & máscara
0	0	0 (limpo)
0	1	0 (sem alteração)
1	0	0 (limpo)
1	1	1 (sem alteração)

Outras operações: colocar bits com valor 0 ⁽²⁾

- Como colocamos a 0 todos os bits dum **char**?

```
// a=operando, m=máscara
unsigned char m = 0x00;
a = a & m;
```

```
a: 0011 1011
& m: 0000 0000
-----
a&m: 0000 0000
```

- Como colocamos a 0 os 2 bits menos significativos (posições 1 e 0) de a sem alterar os restantes bits?

```
unsigned char m = 0xFC;
a = a & m;
```

```
a: 0011 1011
& m: 1111 1100
-----
a&m: 0011 1000
```

// 02-set-reset-bits.c

Outras operações: colocar bits com valor 1 ⁽¹⁾

➤ Usando o operador **|**:

Efetuar a operação **OU** (**|**) entre o operando e '1' coloca os bits do operando a 1, enquanto o **OU** entre o operando e '0' não altera os bits do operando

➤ Ou seja, definimos uma máscara com 1's nas posições onde queremos colocar o operando a 1 e 0's nas outras posições

bit do operando	bit da máscara	operando máscara
0	0	0 (sem alteração)
0	1	1 (colocado a 1)
1	0	1 (sem alteração)
1	1	1 (colocado a 1)

Outras operações: colocar bits com valor 1 ⁽²⁾

- Como colocamos a 1 todos os bits dum **char**?

```
// a=operando, m=máscara
unsigned char m = 0xFF;
a = a | m;
```

```
a: 0011 1010
| m: 1111 1111
-----
a|m: 1111 1111
```

- Como colocamos a 1 os 2 bits mais significativos (posições 7 e 6) de a sem alterar os restantes bits?

```
unsigned char m = 0xC0;
a = a | m;
```

```
a: 0011 1010
| m: 1100 0000
-----
a|m: 1111 1010
```

// 02-set-reset-bits.c

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - **inverter os bits**
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Outras operações: inverter os bits ⁽¹⁾

- Utilizando o operador $\wedge$:
 - Efetuar a operação XOR ($\wedge$) entre o operando e '1' inverte os bits do operando ($0 \rightarrow 1, 1 \rightarrow 0$), enquanto o XOR entre o operando e '0' não altera o operando
- Ou seja, definimos uma máscara com 1's nas posições onde queremos inverter o operando e 0's nas outras posições

bit do operando	bit da máscara	operando $\wedge$ máscara
0	0	0 (sem alteração)
0	1	1 (inverteu)
1	0	1 (sem alteração)
1	1	0 (inverteu)

Outras operações: inverter os bits (2)

- Como invertemos todos os bits dum **char**?

```
// a=operando, m=máscara
unsigned char m = 0xFF;
a = a ^ m;
a = ~a; // também funciona
```

```
a: 0011 1011
^ m: 1111 1111
-----
a^m: 1100 0100
```

- Como invertemos os 2 bits menos significativos (posições 1 e 0) de a sem alterar os restantes bits?

```
unsigned char m = 0x03;
a = a ^ m;
```

```
a: 0011 1010
^ m: 0000 0011
-----
a^m: 0011 1001
```

// 02-inverter-bits.c

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - **verificar se os bits estão a 0|1**
 - contar os bits que estão a 0|1
 - copiar bits de uma posição para outra

Outras operações: verificar se os bits são '1' ⁽¹⁾

➤ Usando o operador &:

1. Efetuamos a operação **E (&)** entre o operando e uma máscara com 1's nas posições que queremos verificar e 0's nas outras posições
2. Verificamos se o **resultado** da operação **&** entre o operando e máscara é **igual à máscara**

bit do operando	bit da máscara	operando & máscara
0	0	0
0	1	0 (difere da máscara=> é 0)
1	0	0
1	1	1 (igual à máscara => é 1)

Outras operações: verificar se os bits são '1' ⁽²⁾

- Como verificamos se todos os bits dum **char** são 1?

```
// a=operando, m=máscara
unsigned char m = 0xFF;
if ((a & m) == m)
    ...
```

```
    a: 0011 1010
    & m: 1111 1111
    -----
    a&m: 0011 1010
```

- Como testamos se os 2 bits menos significativos (posições 1 e 0) do operando a são 1's?

```
unsigned char m = 0x03;
if ((a & m) == m)
    ...
```

```
    a: 0011 1010
    & m: 0000 0011
    -----
    a&m: 0000 0010
```

// 02-testar-bits1.c

a & m ≠ m ←

Outras operações: verificar se os bits são '0'

- Como verificar se todos os bits dum **char** estão a 0?

???

- Como verificar se os bits nas posições 3 e 2 dum **char** são 0's?

???

Conteúdo

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - **contar os bits que estão a 0|1**
 - copiar bits de uma posição para outra

Outras operações: contar os bits que são '1' ⁽¹⁾

- Usando os operadores do C:
 1. aplicamos o procedimento que vimos (nos slides anteriores) para testar se o bit na posição *i* está a '1'
 2. repetimos o passo 1 para cada um dos bits do operando, um bit de cada vez
 3. cada vez que o bit estiver a '1', somamos 1 ao valor dum contador (que começa em zero)
- Máscara a usar no teste do bit *i*:
 $m = 1 \ll i;$
máscara com '1' na posição *i* e '0' nas outras posições

Outras operações: contar os bits que são '1' ⁽²⁾

- Como descobrimos quantos bits estão a 1 num `unsigned char`?

```
int            i;  
unsigned char  m, a=176;  
unsigned int   cnt = 0;  
for (i=0; i<8; i++) {  
    m = 1 << i;  
    if ((a & m) == m)  
        cnt += 1;  
}
```

Conteúdo

- Expressões
- Operadores
 - Com um operando
 - Com dois operandos
 - Relacionais
 - Lógicos
 - Atribuição
- Precedência dos operadores e ordem de avaliação em expressões
- Manuseamento de bits
- Operadores ao nível do bit: complemento, E, OU e OU-exclusivo
- Operadores para deslocamento de bits
- Outras operações sobre bits
 - colocar bits a 0|1
 - inverter os bits
 - verificar se os bits estão a 0|1
 - contar os bits que estão a 0|1
 - **copiar bits de uma posição para outra**

Outras operações: copiar bits em posições selecionadas de **b** para **a**

- colocar a 0 todos os bits de **a** que vamos substituir -> **partA**
- colocar a 0 todos os bits de **b** que não queremos copiar -> **partB**
- efetuar a operação **OU** entre **partA** e **partB** para obter o resultado final

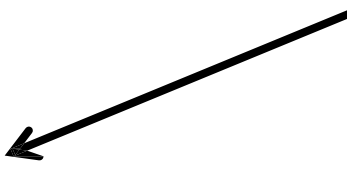
Exemplo: copiar os bits 1:0 de **b** para **a**

a: 0011 1110
& ~m: 1111 1100

b: 1010 0101
& m: 0000 0011

0011 1100

0000 0001



partA = a & ~m

partB = b & m

0011 1100
| 0000 0001

0011 1101

a = partA | partB

Exercícios 1 e 2: operadores

1. O que é escrito pelo código seguinte?

```
int a = 32, b = 5;  
printf("%d %d\n", a--, ++b);  
printf("%d %d\n", --a, ++b);
```

2. Qual é o valor de **a** após executar o código seguinte? E a condição é VERDADEIRA ou FALSA?

```
int a=32, b=5, c=8, d=4, e=12;  
if (a -= ((b > c) || (e / d)) + 6)  
    ... fazer uma ação ...
```

Exercícios 3-5: operadores ao nível do bit

3. Escrever código para colocar a 1 os bits 5:2 do operando **a** (do tipo **char**), sem alterar os outros bits
4. Escrever código para copiar os bits 5:2 de **c** para **d**, sem alterar os outros bits. Qual o valor de **d** (em binário) depois da cópia?

```
unsigned char c = 0xd6, d = 0x6c;
```

5. Qual é o valor de **b**, em binário e em hexadecimal, depois de executar o código seguinte?

```
b = 0x1D;  
m = 0xB7;  
b = b ^ m;
```